Departamento de Ingeniería de Sistemas Telemáticos
Ingeniería de Sistemas y Servicios Telemáticos

DOCUMENTO DE DISEÑO SOFTWARE

SDD (Software Design Document)

Nombre del documento	Documento de Diseño Software
Nombre del fichero:	*ISST-SDD-GRUPO07.docx*
Versión:	*V1*
Fecha de entrega:	*28/02/2024*
Caso de estudio:	**ISST-Grupo07-DIY4Rent**
Entrega:	*Sprint 1*

Autores/as:
Andrés Roques, Gerardo Zapico, Nerio Messino

Revisor/a:
Raúl Sánchez

Resumen

Documento que describe el diseño del sistema software (arquitectura, modelo de datos e interfaz de usuario) que se implantará en el caso de estudio DIY4Rent.

Comentarios

INDICE

1 INTRODUCCIÓN 2

2 ANÁLISIS DE LA ARQUITECTURA 2

2.1 Requisitos y arquitectura 2
2.2 Patrón de arquitectura seleccionado 2
2.3 Otros criterios 2

3 DESCRIPCIÓN DE LA ARQUITECTURA 2

3.1 Vista lógica de la arquitectura 2
3.2 Vista de despliegue de la arquitectura 2

4 MODELO DE DATOS 2

5 INTERFAZ DE USUARIO 3

5.1 Estructura y contenido 3
5.2 Flujos de navegación 4

1 INTRODUCCIÓN

El objetivo principal de este documento es el de describir el diseño del sistema software.

Principalmente definiremos la arquitectura del sistema SW y haremos un modelado de datos que nos permita visualizar los elementos principales y sus relaciones. Finalmente diseñaremos una interfaz de usuario incluyendo los elementos visuales con los que los usuarios podrán interactuar mediante flujos de navegación.

2 ANÁLISIS DE LA ARQUITECTURA

Partiendo de la idea principal del caso de estudio como un marketplace web, podemos asumir que la arquitectura se asemeja a la de una aplicación web diseñada para soportar este tipo de negocio. Desde el punto de vista funcional, la arquitectura base de estos sistemas está compuesta por 3 capas: la capa del frontend (que soportará gran parte de la interfaz de usuario), la capa de servicios RESTful (imprescindible en una arquitectura cliente-servidor) y por último la capa del backend (donde se aloja la lógica de la aplicación).

Habiendo estudiado la arquitectura de sistemas parecidos (negocios que han tenido éxito en sus decisiones de modelado SW) se llega a la conclusión de que esta **estructura cliente-servidor** es óptima para cubrir los requisitos del tipo de marketplace DIY4Rent. Algunos factores que han influido en la decisión han sido la amplitud de implementación en negocios existentes y por supuesto las similitudes con los conocimientos del equipo de desarrollo.

Los **criterios analizados** para elegir esta arquitectura son los siguientes:

- El sistema completo debe cubrir todas los requisitos funcionales y no funcionales, así como cumplir con las reglas de dominio impuestas en el documento de visión.
- El desarrollo de la aplicación debe ser abordable por el equipo y escalable a una posible situación extendida del caso de estudio.
- Todas las tecnologías que soporten el desarrollo de esta arquitectura deben ser *Open Source*, ya que esta iniciativa reduce el coste de inversión inicial para los clientes y el equipo de desarrollo.
- Por supuesto, la arquitectura debe estar basada en una solución ampliamente documentada y extendida en el mundo del desarrollo SW para simplificar el estudio de las tecnologías a utilizar por el equipo.

En base a estos criterios hemos tomado las siguientes **decisiones de implementación**:

- Frontend: Nos permitirá diseñar una interfaz de usuario acorde a nuestras condiciones impuestas por el caso de estudio. Permitirá fluidez e interacción entre el usuario y el sistema.
- APIs RESTful: Estas APIs (Application Programming Interface) serán cruciales en la comunicación e interacción entre el cliente (navegador del usuario) y el servidor de la aplicación. Sin ellas, no podría existir comunicación entre procesos complejos como la reserva de una herramienta, el estudio de la disponibilidad, la función de filtrado por características, etc …
- Backend: Es en esta capa del sistema donde se encontrará la lógica (funciones, métodos, variables, etc …) que darán vida al sistema e implementarán gran parte de los requisitos funcionales descritos en el documento de visión.

Para complementar a la arquitectura cliente-servidor, hemos optado por integrar el **diseño Modelo-Vista-Controlador (MVC)** a nuestro contexto.

El **patrón MVC** se apoyará en la arquitectura de la siguiente manera:

- Capa servidor:
  - El Modelo será la representación de los datos y la lógica del negocio y gestionará los objetos principales (usuario final, anunciantes, herramienta, …)
  - El Controlador será responsable de manejar las solicitudes HTTP e interactuar con el Model para preparar los datos que serán enviados a la Vista.
- Capa cliente:
  - La Vista será la interfaz de usuario con la que interactúan los usuarios de la aplicación.
- Comunicación entre capas:
  - La comunicación se realizará por llamadas a servicios RESTful, cuyas solicitudes entrantes serán respondidas por el Controlador con los datos necesarios para actualizar la Vista.

Todos estos componentes se adecúan e integran a los objetivos que DIY4Rent planea cubrir y complementa fuertemente las características del caso de estudio como se ha justificado anteriormente.

A continuación se quiere dar un enfoque inicial de las tecnologías que se utilizarán en cada una de las capas. Estas podrán ser sustituidas o complementadas por otras en los primeros pasos del desarrollo si el equipo se enfrenta a una incompatibilidad o problemática no prevista en el Spring 1 siempre y cuando no entre en conflicto con las reglas de dominio restricciones y se sigan cubriendo los requisitos funcionales y no funcionales:

- **Server-side**
  - **Spring Boot**: Framework desarrollado en Java diseñado para simplificar el desarrollo de aplicaciones web basadas en Spring. Algunas de las sub-aplicaciones que se integran en este framework y las funcionalidades que cubrirán son:
    - **Spring MVC** servirá como framework en cuanto al patrón de diseño, poniendo a disposición los controladores (DispatcherServlet) que interceptarán las solicitudes HTTP realizadas por el cliente, el modelo que representa los datos y la lógica de negocio y por último la gestión de la vista (posiblemente respuestas JSON consumidas por el framework del lado cliente)
    - **Spring Data JPA** (Java Persistence API) para la gestión e interacción con la BBDD. A diferencia de JDBC (Java Database Connectivity), esta API ofrece una abstracción de más alto nivel y simplifica el acceso a objetos dentro de la BBDD además de

permitir automatizar ciertos procesos de mapeo (objetos-relaciones) y tareas repetitivas.
        - **Spring Security** para la gestión de usuarios y cuentas con características de autenticación, autorización y gestión de sesiones.
        - **Maven** como gestor de dependencias y ciclos de vida de construcción del proyecto Java.
        - **Tomcat** será el servidor web (habitualmente embedido en el framework Spring) y contenedor servlet que proporciona un entorno de ejecución para nuestra aplicación web.
    - **MySQL**: Base de datos relacional encargada de mantener la persistencia y gestión de los datos de la aplicación. Se comunicará con el framework Spring a través de Spring Data JPA. En ella se definirán tablas para almacenar datos y se establecerán relaciones entre ellas, las cuales se especificarán en el proceso de modelado de datos.
- **Client-side**
    - **React framework**: Este extendido framework de desarrollo de frontend servirá para construir la interfaz de usuario de la aplicación web. Permite el desarrollo de una UI interactiva, dinámica y modularizada en componentes, con la posibilidad de gestionar el estado interno de cada uno de ellos. Interactúa con el backend del servidor a través de peticiones HTTP para enviar y recibir datos.
        - **Axios HTTP Library** para realizar solicitudes CRUD (GET, POST, PUT, DELETE, etc …) mediante promesas JavaScript.
        - **React Router** en el manejo de navegación entre páginas.
        - **Redux** para la gestión del estado de la vista de la aplicación.

En resumen, la arquitectura cliente-servidor proporciona el marco general de la aplicación, mientras que el patrón de diseño MVC se utilizará para organizar y estructurar el código tanto en el lado cliente como en el lado servidor, lo que permitirá un desarrollo más modular, mantenible y escalable del sistema.

2.1 Requisitos y arquitectura

Realizando un enfoque exhaustivo sobre el documento de visión y el caso de estudio podemos destacar varios requisitos y objetivos que tienen un impacto significativo sobre la arquitectura SW. Estos se derivan tanto de las necesidades de los usuarios como de las consideraciones técnicas y legales:

- Escalabilidad y rendimiento: Como en cualquier aplicación web, estos dos requisitos son imprescindibles para manejar un crecimiento sustancial en el número de usuario (a medio-largo plazo). Es por esto que una arquitectura flexible y adaptativa a diferentes niveles de carga asegura un rendimiento óptimo durante periodos de alta demanda.
- Seguridad y protección de datos: Atendiendo a las reglas de dominio impuestas por el caso de estudio, debemos asegurarnos que la arquitectura cumple con la seguridad de la información personal y financiera de los usuarios.
- Integración de pagos y mensajería: los actores secundarios en este caso de estudio no pueden ser olvidados a la hora de diseñar la infraestructura SW ya que deberá existir comunicación con ellos para su uso por parte del usuario.

- Experiencia de usuario: este objetivo es primordial para el éxito del negocio, ya que la aplicación será explotada por usuarios finales que únicamente tendrán opción de interactuar con el sistema a través de la interfaz de usuario. Es por esto que la experiencia ofrecida debe cuidarse y hacer lo más intuitiva y fácil de usar posible. Se priorizará la usabilidad y la navegación fluida para mejorar la experiencia de usuario.
- Cumplimiento legal y normativo: El cumplimiento de las políticas y procedimientos impuestos por las leyes y regulaciones como GDPR y DSA estará garantizado dentro del marco normativo y operaciones de la plataforma.
- Documentación y Mantenimiento: Se mantendrá una documentación exhaustiva del desarrollo y funcionamiento del sistema, junto con registros de actividades de usuario con el objetivo de garantizar la transparencia y trazabilidad prometidas dentro de la plataforma.

2.2 Patrón de arquitectura seleccionado

Después de analizar varios patrones arquitectónicos como:

- Arquitectura orientada a eventos: comunicación entre componentes a través de eventos.
- Arquitectura de microservicios: descomposición de la aplicación en un conjunto de servicios pequeños e independientes, cada uno enfocado a una función o tarea.
- Arquitectura sin servidor (serverless): infraestructura subyacente gestionada por el proveedor en la nube.

Hemos optado por la decisión de desarrollar la aplicación en torno a una arquitectura cliente-servidor monolítica frente a otras opciones por las siguientes razones:

- Desarrollo más rápido: En comparación al desarrollo de microservicios, una implementación monolítica acorta el proceso inicial de despliegue y coordinación de componentes.
- Menor curva de aprendizaje: Este concepto ambiguo es aplicable al equipo en cuanto a la experiencia limitada en construcción y gestión de microservicios.
- Facilidad de mantenimiento: El hecho de centralizar los componentes en un solo lugar hace más fácil las modificaciones y mejoras del servicio.
- Requisitos de escalabilidad: La realidad es que no anticipamos una demanda masiva de escalabilidad en el futuro cercano, con lo cual podremos escalar verticalmente añadiendo recursos a la aplicación inicialmente sin necesidad de tener que escalar horizontalmente.

La decisión de no utilizar patrones como el orientado a eventos o el serverless parte de la diferencia de cualidades respecto a la metodología de desarrollo del equipo y la incapacidad de cubrir ciertos requisitos además de resultar más costosa en algunos escenarios (recursos en la nube).

2.3 Otros criterios y restricciones

Nuestro ecosistema de desarrollo prioriza el seguimiento de mejores prácticas y estándares de la industria SW como convención de nomenclaturas o patrones de diseño comunes (MVC).

Restricciones de compatibilidad impuestas por los servicios externos (pago y mensajería) serán tenidos en cuenta a la hora de utilizar productos de software pre-existentes.

El uso de componentes modulares y bibliotecas compartidas (conjunto de funciones, clases o recursos que se comparten entre múltiples partes de una aplicación) son algunas de las estrategias de reutilización a implementar por el equipo.

Debido a la restricción impuesta por los stakeholders de utilizar el lenguaje de programación Java para el desarrollo de la capa de backend, se hará uso del framework de desarrollo Spring Boot, obligando al equipo a estar familiarizado con el uso de este.

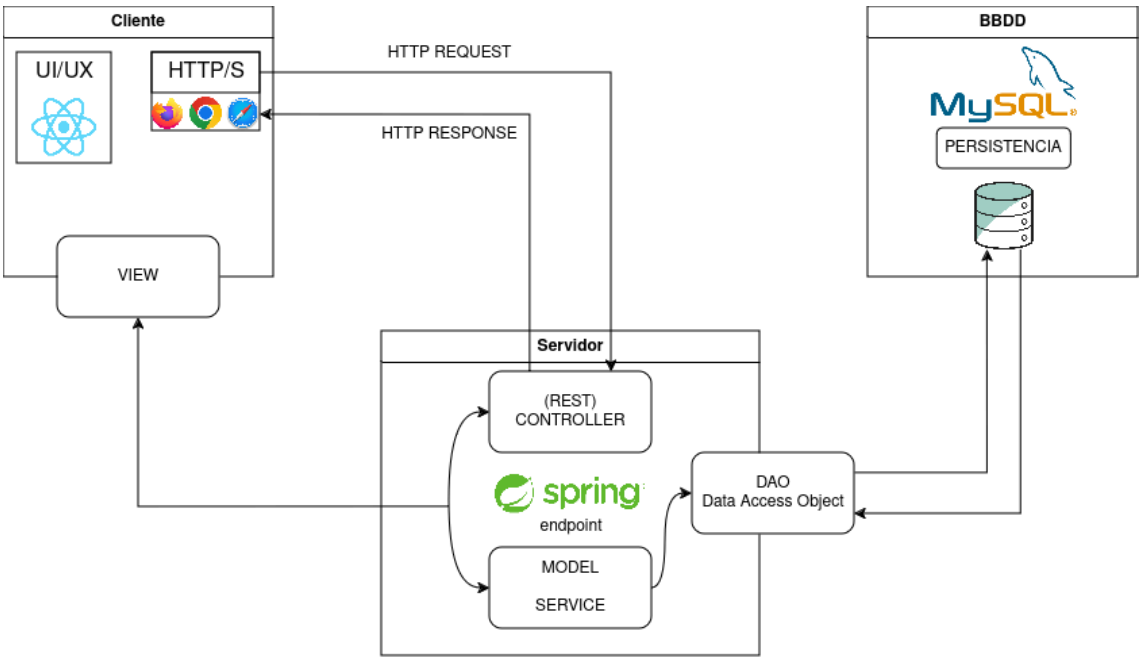
El número de miembros en el equipo (3 miembros del equipo de desarrollo + Scrum Master + Product Owner) y sus capacidades ponen límites a las consideraciones de complejidad en la estructura diseñada. Esto añadido al tiempo disponible para el desarrollo influirá en la profundidad y alcance de la arquitectura, la cual, fuera del marco de la asignatura ISST (al que tenemos que ajustarnos por razones obvias) podría extenderse mucho más allá.

3 DESCRIPCIÓN DE LA ARQUITECTURA

La arquitectura del sistema se basa en un enfoque cliente-servidor monolítico, complementado con el patrón de diseño Modelo-Vista-Controlador (MVC). Esta elección se realizó después de un análisis exhaustivo de varios patrones arquitectónicos y consideraciones específicas del equipo de desarrollo y los requisitos del proyecto. En esta sección se describe el detalle de la arquitectura que llevaremos a cabo durante el proceso de desarrollo de DIY4Rent

Vista lógica de la arquitectura:

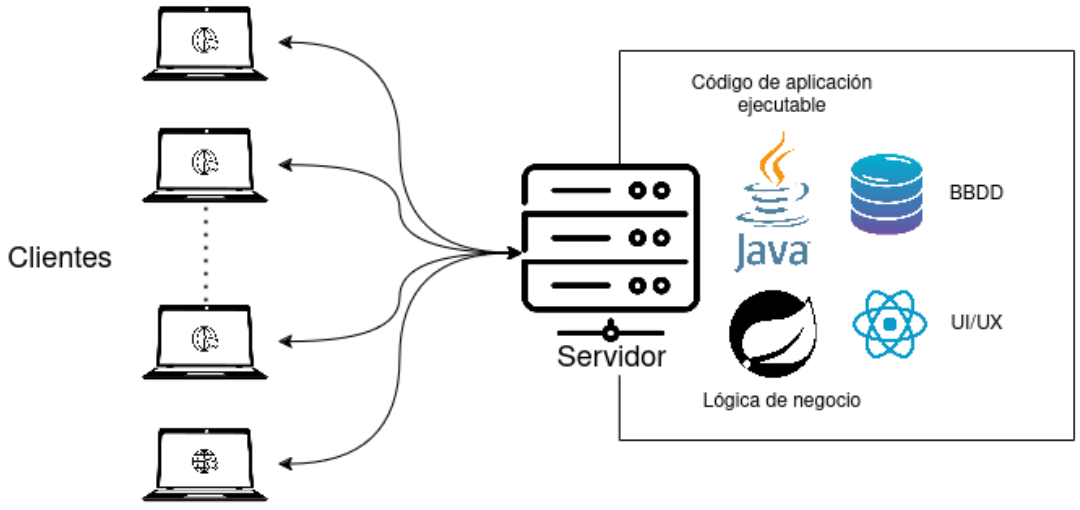
En la vista lógica, el sistema se descompone en tres capas principales: frontend, backend y servicios RESTful. La capa front end se encarga de la interfaz de usuario y la interacción con el usuario final. La capa backend contiene la lógica de negocio y gestiona los datos del sistema. Los servicios RESTful facilitan la comunicación entre el cliente y el servidor. El patrón MVC se implementa en la capa del servidor, donde el Modelo representa los datos y la lógica del negocio, el Controlador maneja las solicitudes HTTP y el envío de datos a la Vista para su presentación al usuario.



Vista de despliegue de la arquitectura:

La vista de despliegue muestra la configuración física del sistema en los nodos de la red. El sistema se despliega en servidores que ejecutan las diferentes partes del sistema, como el backend y los servicios RESTful, mientras que la vista/interfaz de usuario será representada en los navegadores de los usuario de la aplicación. Se garantiza la escalabilidad y el rendimiento del sistema mediante una arquitectura flexible y adaptativa que permite añadir recursos según sea necesario.

En resumen, la arquitectura del sistema se ha diseñado cuidadosamente para cumplir con los requisitos funcionales y no funcionales del proyecto, asegurando un desarrollo rápido, mantenible y escalable del sistema.

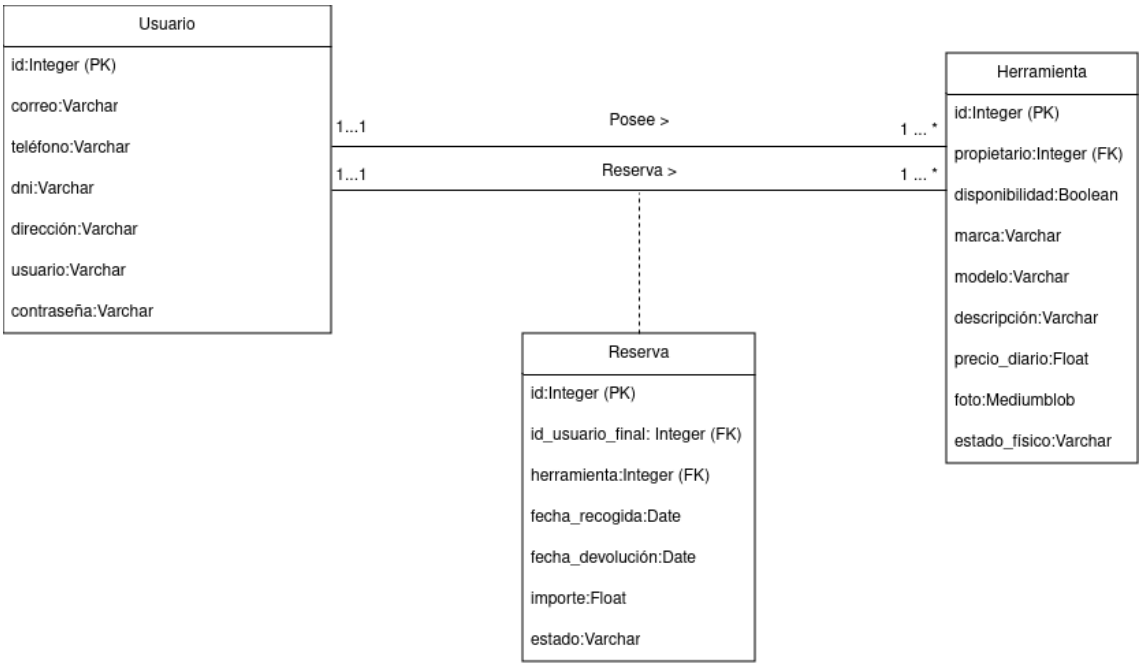


4 MODELO DE DATOS

Habiendo analizado el tipo de arquitectura y sistema que vamos a implementar en el desarrollo del servicio web DIY4Rent, podemos describir el modelado de datos de la siguiente manera.

Las entidades que se han observado durante el desarrollo del servicio son:

- Usuario: En esta “clase” se distinguen los actores principales del caso de estudio, cuya diferenciación está basada en la relación que mantienen con una herramienta. Un mismo usuario del sistema puede alquilar y publicar herramientas simultáneamente. En el diagrama se especifican los atributos de esta entidad.
- Herramienta: Artículo publicado por un usuario anunciante y reservada por un usuario final. Mantiene 2 tipos de relación con el usuario.
- Reserva: Entidad generada tras el acto de reservar una herramienta por un usuario. En ella estarán definidos los atributos necesario para dar información suficiente sobre la relación herramienta-reserva.



Entidad		Atributo							
Nombre	Descripción	Nombre	Descripción	Tipo (Tamaño)	PK	FK	Obligatorio	Unicidad	Verificación
Usuario	Cliente del servicio que busca, solicita y alquila una herramienta	Id	Identificador autoincremental automático por orden cronológico de registro que garantiza la unicidad de los usuarios	INTEGER	✔	**X**	✔	✔	**X**
		Correo	Correo de registro de usuario	VARCHAR (255)	**X**	**X**	✔	✔	✔
		DNI	Documento Nacional de Identidad unico por persona, compuesto por un codigo de 8 numeros y una letra	VARCHAR (255)	**X**	**X**	✔	✔	✔
		Teléfono	Número de teléfono móvil del usuario registrado (9 números)	VARCHAR (255)	**X**	**X**	✔	✔	✔

		Dirección	Direccion del usuario	VARCHAR (255)	**X**	**X**	✔	**X**	**X**
		Usuario	Alias elegido por el usuario al registrarse en la web	VARCHAR (255)	**X**	**X**	✔	✔	✔
		Contraseña	Conjunto de letras, numeros y símbolos elegidos como contraseña del usuario. Debe cumplir unos requisitos para ser aceptada.	VARCHAR (255)	**X**	**X**	✔	**X**	✔
Herramienta	Artículo publicado por un usuario anunciante y reservada por un usuario final. Mantiene 2 tipos de relación con el usuario.	Id	Identificador autoincremental automático por orden cronológico de registro que garantiza la unicidad de las herramientas	INTEGER	✔	**X**	✔	✔	**X**
		Propietario	Nombre y apellidos del usuario registrado	USUARIO	**X**	✔	✔	**X**	**X**
		Disponibilidad	Correo de registro de usuario	BOOLEAN	**X**	**X**	✔	✔	✔
		Marca	Marca de la herramienta publicada	VARCHAR (255)	**X**	**X**	✔	**X**	**X**
		Modelo	Modelo de la marca de la herramienta publicada	VARCHAR (255)	**X**	**X**	✔	**X**	**X**
		Descripción	Breve descripción de la herramienta publicada	VARCHAR (255)	**X**	**X**	✔	**X**	**X**
		Precio diario	Precio asociado al alquiler del artículo por 24h	FLOAT	**X**	**X**	✔	✔	**X**
		Foto del producto	Fotografía que corresponda a la herramienta que posea el usuario anunciante.	MEDIUMBLOB	**X**	**X**	**X**	**X**	✔
		Estado físico	Eleccion del nivel de degradación del estado físico del producto	VARCHAR (255)	**X**	**X**	✔	✔	**X**
Reserva	Entidad generada tras el acto de reservar una herramienta por un usuario. En ella estarán definidos los atributos necesario para dar información suficiente sobre la relación herramienta-reserva.	Id	Identificador autoincremental automático por orden cronológico de registro que garantiza la unicidad de las reservas	INTEGER	✔	**X**	✔	✔	**X**
		Usuario final	Id proveniente de la clase Usuario	INTEGER	**X**	✔	✔	**X**	**X**
		Herramienta	Id proveniente de la clase Herramienta	INTEGER	**X**	✔	✔	✔	**X**
		Fecha de recogida	Hora de la recogida de la herramienta por parte del usuario final	DATE	**X**	**X**	✔	**X**	✔
		Fecha de devolución	Fecha de la devolución de la herramienta por parte del usuario final	DATE	**X**	**X**	✔	**X**	✔
		Importe	Coste total del servicio de prestación de la herramienta	FLOAT	**X**	**X**	✔	**X**	**X**
		Estado	Estado de la reserva que podrá estar pendiente, confirmada o cancelada	VARCHAR (255)	**X**	**X**	✔	**X**	✔

5 INTERFAZ DE USUARIO

5.1 Estructura y contenido

Para nuestra aplicación tendremos 4 pantallas: pantalla de inicio, registro de usuarios o login, pantalla de mis herramientas, pantalla de mis reservas y la de inicio/buscar herramientas. Todas ellas tendrán una barra de navegación en la parte superior de la pantalla para navegar entre ellas. Adicionalmente en esta barra de navegación también estará el apartado de contáctanos.

En la pantalla de inicio tendremos: una sección donde explicaremos qué puede encontrar el usuario en la página, otra sección con una breve descripción acerca de nosotros, un botón para invitarles a registrarse, que les llevará a la pantalla de registro de usuarios. Al final, habrá una sección para que se puedan poner en contacto con nosotros con un campo donde especifique su nombre, otro campo para su correo electrónico y un campo donde pueda describir su consulta.

En la pantalla de registro se presenta a los usuarios un formulario con los campos necesarios para la creación de la cuenta. Si, por otra parte, ya tenían una cuenta se les proporcionará la opción de loguearse, mostrándoles un formulario donde introducir nombre de usuario y contraseña.

En la pantalla de mis herramientas tendremos, en primer lugar, un botón para añadir una nueva herramienta que abrirá un formulario para que el usuario rellene con las características técnicas de la herramienta. A continuación, habrá una lista con todas las herramientas que el usuario haya creado al pulsar el botón anterior. Cada ítem de la lista tendrá dos botones uno para modificar las características de la herramienta o eliminar la herramienta y otro para publicar la herramienta. Por último, habrá una lista con todas las herramientas que el usuario tenga publicadas en el marketplace. Cada ítem de esta lista tendrá un botón para retirar la herramienta de la lista de publicaciones.

Por último, en la pantalla de buscar herramientas se le presenta al usuario un formulario para ajustar los criterios de búsqueda a sus necesidades. Habrá un botón para iniciar la búsqueda, que mostrará una lista con todas las herramientas que cumplan los criterios establecidos.

5.2 Flujos de navegación

